



NANYANG JUNIOR COLLEGE

JC1 END-OF-YEAR EXAMINATION

Higher 2

COMPUTING

9569/02

Paper 2 (Lab-based)

27th September 2024

2 hours

Additional Materials: Removable storage device
 Electronic version of `transactions.txt` data file
 Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **3** marks out of 66 will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [] at the end of each question or part question. The total number of marks for this paper is 66.

This document consists of **8** printed pages.



NANYANG JUNIOR COLLEGE
Internal Examinations

Instruction to candidates:

Your program code and output for each of Task 1 to 3 should be saved in a single `.ipynb` file using Jupyter Notebook. For example, your program code and output for Task 1 should be saved as

`TASK1_<your name>_<class>.ipynb`

Make sure that each of your `.ipynb` files shows the required output in Jupyter Notebook.

1 Name your Jupyter Notebook as:

`TASK1_<your name>_<class>.ipynb`

RGB is a colour model in which the red, green and blue primary colours of light are added together in various ways to reproduce a broad array of colours. Such colours can be represented as a set of six non-case-sensitive hexadecimal characters eg. `1AB7F4`, where the first pair of hexadecimal characters refers to the red colour, the next pair refers to the green colour, and the last pair refers to the blue colour.

This task is to create an interface to:

- obtain an hexadecimal RGB colour code from a user
- validate and store the RGB colour code in a list
- display the list on demand
- convert each of the colour codes in the list to their corresponding decimal representation and write each converted decimal colour code to an output file.

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 1.1
Program code
Output:
```

Task 1.1

Write a function called `task1_1(colour_code)` that returns a boolean value.

The function should:

- validate that the `colour_code` is a six-character string, and each character within the string contains values ranging from 0 to 9 and A to F only
- return `True` if the above rule is met, otherwise return `False`. [4]

Task 1.2

Create **four** tests to fully test your function. Your four input values should be:

- an integer
- a valid string
- a string containing characters, some of which are outside the permitted range
- a value of incorrect length.

Test your function with your four input values by calling it using the following statement:

```
print(task1_1(colour_code) == <outcome>)
```

The four statements should all print `True`.

For example,

```
print(task1_1("1ABF4") == False)
```

Do not use the example provided but create your own.

[4]

Task 1.3

Write a function `task1_3(colour_code)` that returns a tuple.

The function should:

- take in a string `colour_code` representing a six-character hexadecimal RGB colour code
- convert each of the hexadecimal RGB colour pairs Red, Green and Blue into their corresponding decimal representation
- return the decimal RGB colour code as a tuple of 3 integers.

[4]

Task 1.4

Write program code to do the following:

- initialise an empty list for the hexadecimal colour codes
- present the user with a menu, offering the following options:
 - 1) Add a hexadecimal Colour Code
 - 2) View hexadecimal Colour Code List
 - 3) Convert to decimal Colour Code and write to file
 - 4) Exit
- prompt the user to select a numerical option
- loop until a valid option is input
- if **option 1** is selected:
 - prompt the user to input a hexadecimal colour code
 - use your function from **Task 1.1** to validate the colour code
 - add the colour code to the list if it is valid or output an appropriate error.
- if **option 2** is selected:
 - display the hexadecimal colour codes present in the list
- If **option 3** is selected:
 - use your function from **Task 1.3** to convert each hexadecimal colour code in the list
 - write the converted colour code to the file `converted_colours.txt` with each decimal colour code occupying a single line, separated by commas eg. 32, 255, 179. [7]

Test the program in the following sequence:

- Select one invalid option.
- Select option 1 twice, with the corresponding inputs `A21EF9` and `F36C2D`.
- Select option 3. [3]

Save your Jupyter Notebook for Task 1.

(Total: 22)

2 Name your Jupyter Notebook as:

TASK2_<your name>_<class>.ipynb

This task is to perform a sorting algorithm on 80 integers held in a 1-dimensional list and then search for a value in the list.

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 2.1
        Program code
        Output:
```

Task 2.1

Write a function `task2_1(number_limit)` that returns a list.

The function should:

- take in an integer `number_limit`
- initialise a list
- generate 80 random integers between 1 and `number_limit` (inclusive)
- store each integer in the list
- return the list of generated integers.

[3]

Task 2.2

Write a function `task2_2(unsorted_list)` that returns a list.

The function should:

- take in a list `unsorted_list` which represents a list of unsorted integers
- implement an insertion sort algorithm to sort the list in ascending order
- return a new list of sorted integers, without modifying the original list.

Write program code to:

- call your function from **Task 2.1** with a `number_limit` of 100
- call the insertion sort function
- output the sorted list.

[6]

Test your program and show the output.

[1]

Task 2.3

Write a function `task2_3(sorted_list, value)` that returns an integer.

The function should:

- accept `sorted_list` and `value` as parameters:
 - `sorted_list` – a list of sorted integers obtained from **Task 2.2**
 - `value` – an integer identifying the item being searched for
- implement a binary search on the list
- return the position of the `value` in the list (if the `value` appears more than once, then only one position needs to be returned)
- return -1 if the `value` is not in the list.

Write program code to:

- read in an integer value from the user
- call the binary search function with the integer value
- output the returned position if the integer is in the list
- output `Not found` if the integer is not in the list.

[7]

Test the program with the following data:

- largest integer that is in the list
- an integer that is not in the list.

Show your output clearly.

[2]

Save your Jupyter Notebook for Task 2.

(Total: 19)

3 Name your Jupyter Notebook as:

`TASK3_<your name>_<class>.ipynb`

A bank processes financial transactions using a circular queue.

The class `Transaction` will store the following data:

- `amount` – stored as an integer representing cents
- `recipient` – the account ID of the recipient, stored as a string

The `ChequeDeposit` class inherits from `Transaction`, and stores the following additional data:

- `chequeNumber` – stored as an integer

The `ChequeDeposit` class also has the following method defined on it:

- `toString()` – returns a string description of the transaction as follows:
`Deposit cheque 1234567 $60.00 into 567890123`

The `BankTransfer` class inherits from `Transaction`, and stores the following additional data:

- `sender` – the account ID of the sender, stored as a string

The `BankTransfer` class also has the following method defined on it:

- `toString()` – returns a string description of the transaction as follows:
`Transfer $30.00 from 765432109 to 567890123`

The class `Queue` should have the following attributes:

- `data` – an array holding the items in the queue
- `head` – an index representing the position of the next slot that data will be enqueued into, or a suitable value if the queue is full
- `tail` – an index representing the position of the next item to be dequeued, or a suitable value if the queue is empty.

The class `Queue` should have the following methods:

- a constructor that initialises `data`, `head`, and `tail` to appropriate values for an empty queue.
- `enqueue()` takes a `Transaction` as a parameter and inserts it into the queue. It returns `True` if this is successful, or `False` otherwise.
- `dequeue()` removes the first item in the queue and returns the removed item, or `None` if empty.

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 3.1
        Program code
```

Output:

Task 3.1

Write program code to declare the class `Transaction` and its subclasses.

[7]

Task 3.2

Write program code to declare the class `Queue` and its methods.

[8]

Task 3.3

The text file, `transactions.txt`, contains data items for a number of transactions. Each data item is separated by a comma, with each transaction on a new line as follows:

- **transaction ID** e.g. T01 or C03. An ID beginning with 'T' indicates a bank transfer, while an ID beginning with 'C' indicates a cheque deposit
- **amount** involved in the transaction, in cents
- **recipient account ID**
- the **cheque number**, if the transaction is a cheque deposit, or the **sender account ID**, if the transaction is a bank transfer.

Write program code to:

- initialise a new `Queue` of 5 transactions
- read in the information from the text file, `transactions.txt`
- creating an instance of the appropriate subclass for each transaction (either `ChequeDeposit` or `BankTransfer`)
- call `enqueue()` with each new `Transaction` while the queue is not full
- if the queue is full, call `dequeue()` until the queue is empty, displaying a description of each removed transaction using `toString()`.

[7]

Save your Jupyter Notebook for Task 3.

(Total: 22)

----- END OF PAPER -----